

4 Network Containers

4.1 Connecting network namespaces through an IPIP tunnel

4.1.1 Setup

Following the Network Containers lab guide [3], this experiment was performed with two Linux nodes, `ub1` and `ub2`. Each node hosted one Linux network namespace. The objective was to make the namespace in `ub1` communicate with the namespace in `ub2` through an IPIP tunnel between the two physical hosts.

Table 5: Addressing plan used in the IPIP namespace experiment.

Node	Interface / namespace	Address	Role
ub1	ens3	20.0.0.1/24	Physical underlay network
ub1	namespace red	10.10.10.1/24	Namespace endpoint
ub1	veth-red-host	10.10.10.254/24	Gateway for namespace red
ub1	tun-red	172.31.0.1/30	IPIP tunnel interface
ub2	ens3	20.0.0.2/24	Physical underlay network
ub2	namespace blue	10.20.20.1/24	Namespace endpoint
ub2	veth-blue-host	10.20.20.254/24	Gateway for namespace blue
ub2	tun-blue	172.31.0.2/30	IPIP tunnel interface

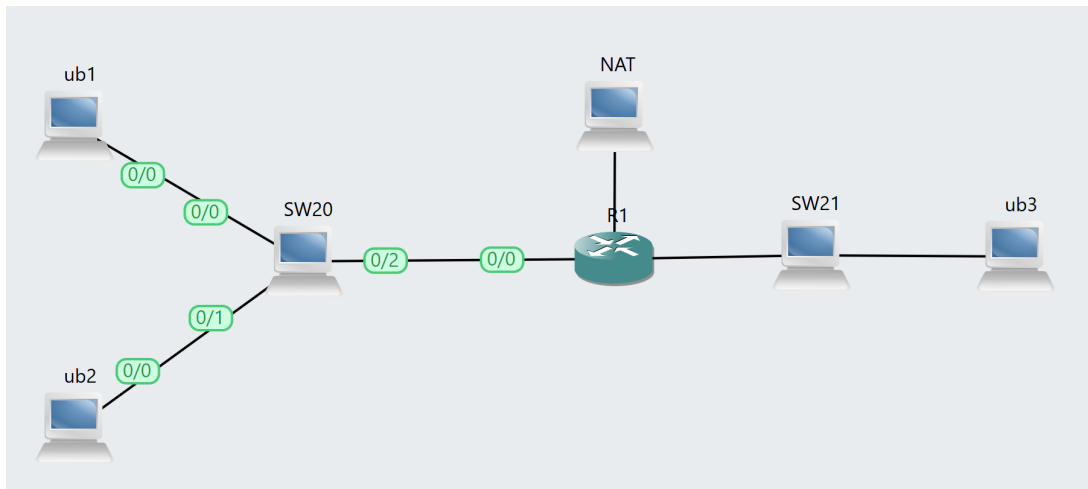


Figure 29: Topology used in Lab 2.3.

4.1.2 IPIP tunnel configuration

On `ub1`, the namespace `red`, the host-side veth interface and the IPIP tunnel were configured as follows:

```
ip netns add red
ip link add veth-red type veth peer name veth-red-host
ip link set veth-red netns red
ip -n red addr add 10.10.10.1/24 dev veth-red
ip -n red link set veth-red up
ip addr add 10.10.10.254/24 dev veth-red-host
ip link set veth-red-host up
ip tunnel add tun-red mode ipip local 20.0.0.1 remote 20.0.0.2
```

```
ip addr add 172.31.0.1/30 dev tun-red
ip link set tun-red up
ip route replace 10.20.20.0/24 dev tun-red
ip netns exec red ip route replace default via 10.10.10.254
sysctl -w net.ipv4.ip_forward=1
iptables -P FORWARD ACCEPT
```

Listing 57: IPIP configuration applied on ub1.

On ub2, the configuration was symmetric:

```
ip netns add blue
ip link add veth-blue type veth peer name veth-blue-host
ip link set veth-blue netns blue
ip -n blue addr add 10.20.20.1/24 dev veth-blue
ip -n blue link set veth-blue up
ip addr add 10.20.20.254/24 dev veth-blue-host
ip link set veth-blue-host up
ip tunnel add tun-blue mode ipip local 20.0.0.2 remote 20.0.0.1
ip addr add 172.31.0.2/30 dev tun-blue
ip link set tun-blue up
ip route replace 10.10.10.0/24 dev tun-blue
ip netns exec blue ip route replace default via 10.20.20.254
sysctl -w net.ipv4.ip_forward=1
iptables -P FORWARD ACCEPT
```

Listing 58: IPIP configuration applied on ub2.

The FORWARD policy was explicitly set to ACCEPT, because Docker may create firewall rules that interfere with forwarding between the namespace and the tunnel interface.

4.1.3 Connectivity validation

Connectivity was tested from namespace **red** to namespace **blue**. The ping succeeded with four replies and no packet loss. The tunnel counters also increased, confirming that the traffic crossed the IPIP tunnel.

```
Output - Exerc?cio 5.3: ping entre namespaces via IPIP

ip netns exec red ping -c 4 -W 2 10.20.20.1; ip -s tunnel show tun-red; printf '\nYEND177980522318:%s\n' $?
PING 10.20.20.1 (10.20.20.1) 56(84) bytes of data.
 64 bytes from 10.20.20.1: icmp_seq=1 ttl=62 time=9.91 ms
 64 bytes from 10.20.20.1: icmp_seq=2 ttl=62 time=1.63 ms
 64 bytes from 10.20.20.1: icmp_seq=3 ttl=62 time=2.00 ms
 64 bytes from 10.20.20.1: icmp_seq=4 ttl=62 time=1.63 ms
4 packets transmitted, 4 received, 0% packet loss, time 3013ms
tun-red: ip/ip remote 20.0.0.2 local 20.0.0.1 ttl inherit
RX: Packets  Bytes      Errors CsumErrs OutOfSeq Mcasts
TX: Packets  Bytes      Errors DeadLoop NoRoute  NoBufs
```

Figure 30: Ping between namespaces and IPIP tunnel counters.

As shown in Figure 30, the namespace **red** reached 10.20.20.1. The tunnel statistics show transmitted and received packets, which confirms that the communication did not remain local to the host.

Wireshark analysis

A Wireshark capture was taken on the physical link of ub1. The capture shows an outer packet between 20.0.0.1 and 20.0.0.2 using IPIP, carrying the inner ICMP packet between 10.10.10.1 and 10.20.20.1. In the outer IP header, the protocol field is 4, which identifies IP-in-IP encapsulation.

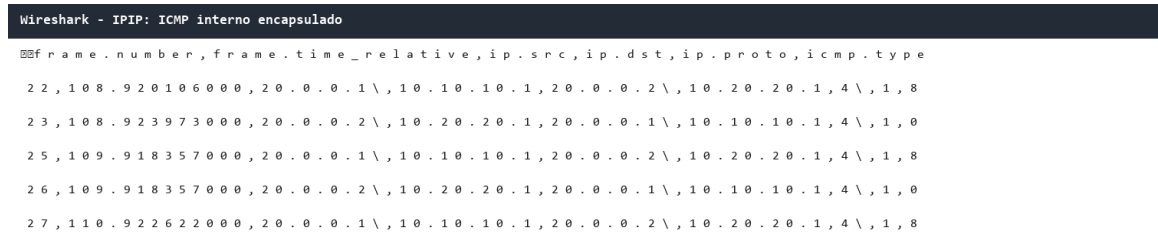


Figure 31: ICMP packet between namespaces encapsulated inside IPIP.

This confirms that the two namespaces, although located in different Linux hosts, communicated through an IPIP tunnel over the physical underlay network.

4.2 Macvlan networks with VLANs

4.2.1 Setup

Two `macvlan` networks were created over VLAN subinterfaces. The blue network used VLAN 10 and the red network used VLAN 20. The purpose of the experiment was to show that containers connected to the same logical network can communicate, while containers connected to different VLAN-backed networks remain isolated.

Table 6: Addressing plan used in the macvlan/VLAN experiment.

Node	Container	Address	Network
ub1	b1	10.60.10.10/24	bluevlan, VLAN 10
ub2	b2	10.60.10.20/24	bluevlan, VLAN 10
ub1	r1	10.60.20.10/24	redvlan, VLAN 20
ub2	r2	10.60.20.20/24	redvlan, VLAN 20

4.2.2 VLAN and macvlan configuration

On both Linux nodes, VLAN subinterfaces were created over `ens3`. Docker `macvlan` networks were then attached to the corresponding VLAN interfaces.

```
ip link add link ens3 name ens3.10 type vlan id 10
ip link add link ens3 name ens3.20 type vlan id 20
ip link set ens3.10 up
ip link set ens3.20 up

docker network create -d macvlan --subnet=10.60.10.0/24 -o parent=ens3.10 bluevlan
docker network create -d macvlan --subnet=10.60.20.0/24 -o parent=ens3.20 redvlan
```

Listing 59: VLAN subinterfaces and macvlan networks.

The containers were then created with fixed addresses:

```
# ub1
docker run -dit --name b1 --network bluevlan --ip 10.60.10.10 alpine sleep 3600
docker run -dit --name r1 --network redvlan --ip 10.60.20.10 alpine sleep 3600

# ub2
docker run -dit --name b2 --network bluevlan --ip 10.60.10.20 alpine sleep 3600
docker run -dit --name r2 --network redvlan --ip 10.60.20.20 alpine sleep 3600
```

Listing 60: Containers attached to the VLAN-backed macvlan networks.

4.2.3 Connectivity and isolation tests

The same-network tests succeeded: `b1` reached `b2` and `r1` reached `r2`. The cross-network test from `b1` to `r2` failed with packet loss, confirming that the two `macvlan` networks were isolated.

```
Output - Exerc?cio 6.4: conectividade e isolamento

PING 10.60.10.20 (10.60.10.20): 56 data bytes
3 packets transmitted, 3 packets received, 0% packet loss
PING 10.60.20.20 (10.60.20.20): 56 data bytes
3 packets transmitted, 3 packets received, 0% packet loss
PING 10.60.20.20 (10.60.20.20): 56 data bytes
3 packets transmitted, 0 packets received, 100% packet loss
default via 10.60.10.1 dev eth0
default via 10.60.20.1 dev eth0
```

Figure 32: Connectivity inside each VLAN and isolation between different VLANs.

Wireshark analysis

The Wireshark capture confirms the presence of 802.1Q VLAN tags. Traffic from the blue network used VLAN ID 10, while traffic from the red network used VLAN ID 20.

Wireshark - macvlan com VLAN 10 e VLAN 20										
frame.number	frame.time.relative	vlan.id	eth.src	eth.dst	ip.src	ip.dst	icmp.type			
35,23	.436934000	10	02:42:0a:3c:0a:0a	02:42:0a:3c:0a:14	10.60.10.10	10.60.10.20	8			
36,23	.438026000	10	02:42:0a:3c:0a:14	02:42:0a:3c:0a:0a	10.60.10.20	10.60.10.10	0			
40,24	.438938000	10	02:42:0a:3c:0a:0a	02:42:0a:3c:0a:14	10.60.10.10	10.60.10.20	8			
41,24	.439787000	10	02:42:0a:3c:0a:14	02:42:0a:3c:0a:0a	10.60.10.20	10.60.10.10	0			
43,25	.441847000	10	02:42:0a:3c:0a:0a	02:42:0a:3c:0a:14	10.60.10.10	10.60.10.20	8			
44,25	.442357000	10	02:42:0a:3c:0a:14	02:42:0a:3c:0a:0a	10.60.10.20	10.60.10.10	0			
47,25	.682466000	20	02:42:0a:3c:14:0a	02:42:0a:3c:14:14	10.60.20.10	10.60.20.20	8			
48,25	.692985000	20	02:42:0a:3c:14:14	02:42:0a:3c:14:0a	10.60.20.20	10.60.20.10	0			
49,26	.674526000	20	02:42:0a:3c:14:0a	02:42:0a:3c:14:14	10.60.20.10	10.60.20.20	8			
50,26	.675163000	20	02:42:0a:3c:14:14	02:42:0a:3c:14:0a	10.60.20.20	10.60.20.10	0			
52,27	.676694000	20	02:42:0a:3c:14:0a	02:42:0a:3c:14:14	10.60.20.10	10.60.20.20	8			
53,27	.677247000	20	02:42:0a:3c:14:14	02:42:0a:3c:14:0a	10.60.20.20	10.60.20.10	0			

Figure 33: Wireshark capture showing VLAN IDs 10 and 20.

The capture confirms that containers belonging to the same logical network use the same VLAN ID, and that separation between the two container networks is achieved through VLAN tagging.

4.3 Linux VXLAN with multicast routing

4.3.1 Setup

In this experiment, two VXLAN-based overlay networks were configured over the physical topology. The blue overlay used VNI 33 and multicast group 239.1.1.3. The red overlay used VNI 66 and multicast group 239.1.1.6. The router R1 was configured with multicast routing and PIM dense mode so that VXLAN multicast traffic could cross the routed underlay.

Table 7: VXLAN overlay networks used in the experiment.

Overlay	Subnet	VNI	Multicast group
bluenet	10.0.0.0/24	33	239.1.1.3
rednet	11.0.0.0/24	66	239.1.1.6

Table 8: Container distribution used in the VXLAN multicast experiment.

Node	Blue containers	Red containers	Role
ub1	b1: 10.0.0.1, b2: 10.0.0.2	r1: 11.0.0.1, r2: 11.0.0.2	Blue and red VTEP
ub2	b3: 10.0.0.3, b4: 10.0.0.4	r3: 11.0.0.3, r4: 11.0.0.4	Blue and red VTEP
ub3	b5: 10.0.0.5, b6: 10.0.0.6	none	Blue VTEP only

4.3.2 VXLAN configuration

The following configuration was applied on ub1. The same logic was applied on ub2; on ub3, only the blue VXLAN interface was configured.

```
ip link add blue type vxlan id 33 group 239.1.1.3 ttl 5 dev ens3 dstport 4789
ip link add red type vxlan id 66 group 239.1.1.6 ttl 5 dev ens3 dstport 4789
ip link set blue up
ip link set red up
ip addr add 10.0.0.101/24 dev blue
ip addr add 11.0.0.101/24 dev red

docker network create -d macvlan --subnet=10.0.0.0/24 --gateway=10.0.0.101 -o parent=blue
    bluenet
docker network create -d macvlan --subnet=11.0.0.0/24 --gateway=11.0.0.101 -o parent=red
    rednet
```

Listing 61: VXLAN and Docker network configuration on ub1.

R1 was configured to forward multicast traffic between the underlay networks:

```
ip multicast-routing
interface FastEthernet0/0
    ip pim dense-mode
interface FastEthernet0/1
    ip pim dense-mode
```

Listing 62: Multicast routing configuration on R1.

4.3.3 Connectivity tests

The required ICMP tests were executed successfully. The tested paths included local communication within a node, communication between **ub1** and **ub2**, communication between **ub1** and **ub3** through **R1**, and communication in the red overlay.

```
Output - Exerc?cio 6.5: pings VXLAN blue/red

PING 10.0.0.2 (10.0.0.2): 56 data bytes
3 packets transmitted, 3 packets received, 0% packet loss
PING 10.0.0.3 (10.0.0.3): 56 data bytes
3 packets transmitted, 3 packets received, 0% packet loss
PING 10.0.0.6 (10.0.0.6): 56 data bytes
3 packets transmitted, 3 packets received, 0% packet loss
PING 11.0.0.3 (11.0.0.3): 56 data bytes
3 packets transmitted, 3 packets received, 0% packet loss
10.0.0.3 dev eth0 lladdr 02:42:0a:00:00:03 ref 1 used 0/0/0 probes 4 REACHABLE
10.0.0.2 dev eth0 lladdr 02:42:0a:00:00:02 ref 1 used 0/0/0 probes 4 REACHABLE
11.0.0.3 dev eth0 lladdr 02:42:0b:00:00:03 ref 1 used 0/0/0 probes 4 REACHABLE
```

Figure 34: ICMP tests over the blue and red VXLAN overlays.

The successful pings show that **bluenet** operated across the three nodes, while **rednet** operated between **ub1** and **ub2**.

IGMP and multicast routing

R1 registered the expected multicast groups, as verified with `show ip igmp groups` and the multicast routing table. The blue group, **239.1.1.3**, appeared on both underlay sides because **ub1/ub2** and **ub3** had blue VTEPs. The red group, **239.1.1.6**, appeared only where red VTEPs existed.

```
Output - R1: IGMP e multicast routing

*Mar  1 00:00:40.731: %PIM-5-DRCHG: DR change from neighbor 0.0.0.0 to 20.0.0.254 on interface FastEthernet0/0
*Mar  1 00:00:40.763: %PIM-5-DRCHG: DR change from neighbor 0.0.0.0 to 21.0.0.254 on interface FastEthernet0/1
show ip interface brief
FastEthernet0/0      20.0.0.254      YES manual up      up
FastEthernet0/1      21.0.0.254      YES manual up      up
C      21.0.0.0 is directly connected, FastEthernet0/1
C      20.0.0.0 is directly connected, FastEthernet0/0
20.0.0.254      FastEthernet0/0      v2/D    0      30      1      20.0.0.254
21.0.0.254      FastEthernet0/1      v2/D    0      30      1      21.0.0.254
224.0.1.40      FastEthernet0/0      00:00:06 00:02:53 20.0.0.254
FastEthernet0/0      20.0.0.254      YES manual up      up
FastEthernet0/1      21.0.0.254      YES manual up      up
C      21.0.0.0 is directly connected, FastEthernet0/1
C      20.0.0.0 is directly connected, FastEthernet0/0
239.1.1.3      FastEthernet0/1      00:00:26 00:02:35 21.0.0.1
239.1.1.3      FastEthernet0/0      00:00:32 00:02:33 20.0.0.2
239.1.1.6      FastEthernet0/0      00:00:32 00:02:35 20.0.0.2
224.0.1.40      FastEthernet0/0      01:53:06 00:02:23 20.0.0.254
R1#show ip mroute 239.1.1.3
Outgoing interface flags: H - Hardware switched, A - Assert winner
(*, 239.1.1.3), 00:00:34/stopped, RP 0.0.0.0, flags: DC
  Outgoing interface list:
    FastEthernet0/1, Forward/Dense, 00:00:27/00:00:00
    FastEthernet0/0, Forward/Dense, 00:00:34/00:00:00
(20.0.0.1, 239.1.1.3), 00:00:34/00:02:54, flags: T
  Incoming interface: FastEthernet0/0, RPF nbr 0.0.0.0
  Outgoing interface list:
```

Figure 35: IGMP groups and multicast routing state observed on **R1**.

This output confirms the role of IGMP in registering the multicast receivers and the role of multicast routing in forwarding VXLAN traffic across the routed underlay.

VXLAN packet analysis

The Wireshark capture shows IGMP joins, ARP traffic and ICMP traffic transported over UDP/4789. The VXLAN header contains the VNI, which identifies the Layer 2 overlay to which the encapsulated frame belongs.

Wireshark - VXLAN multicast, ARP e ICMP

```

7,23.416938000,IGMPv2,20.0.0.254,224.0.0.1,,,,,
9,32.105800000,IGMPv2,20.0.0.254,224.0.1.40,,,,,
10,35.757482000,IGMPv2,20.0.0.1,239.1.1.6,,,,,
11,35.757482000,IGMPv2,20.0.0.1,239.1.1.3,,,,,
23,37.108570000,IGMPv2,20.0.0.1,239.1.1.3,,,,,
25,39.333850000,IGMPv2,20.0.0.2,239.1.1.6,,,,,
26,39.333850000,IGMPv2,20.0.0.2,239.1.1.3,,,,,
42,42.836954000,IGMPv2,20.0.0.2,239.1.1.3,,,,,
45,44.119231000,IGMPv2,20.0.0.2,239.1.1.6,,,,,
54,49.513816000,ARP,20.0.0.1,239.1.1.3,4789,33,10.0.0.1,10.0.0.2,
57,51.891657000,ARP,20.0.0.1,239.1.1.3,4789,33,10.0.0.1,10.0.0.3,
58,51.893686000,ARP,20.0.0.2,20.0.0.1,4789,33,10.0.0.3,10.0.0.1,
59,51.895259000,ICMP,20.0.0.1\,10.0.0.1,20.0.0.2\,10.0.0.3,4789,33,,8
60,51.896789000,ICMP,20.0.0.2\,10.0.0.3,20.0.0.1\,10.0.0.1,4789,33,,0
62,52.895736000,ICMP,20.0.0.1\,10.0.0.1,20.0.0.2\,10.0.0.3,4789,33,,8
63,52.903325000,ICMP,20.0.0.2\,10.0.0.3,20.0.0.1\,10.0.0.1,4789,33,,0
64,53.895606000,ICMP,20.0.0.1\,10.0.0.1,20.0.0.2\,10.0.0.3,4789,33,,8
65,53.896713000,ICMP,20.0.0.2\,10.0.0.3,20.0.0.1\,10.0.0.1,4789,33,,0
66,54.037443000,ARP,20.0.0.1,239.1.1.3,4789,33,10.0.0.2,10.0.0.6,
67,54.057634000,ARP,21.0.0.1,20.0.0.1,4789,33,10.0.0.6,10.0.0.2,
68,54.058669000,ICMP,20.0.0.1\,10.0.0.2,21.0.0.1\,10.0.0.6,4789,33,,8
69,54.079974000,ICMP,21.0.0.1\,10.0.0.6,20.0.0.1\,10.0.0.2,4789,33,,0
70,55.041736000,ICMP,20.0.0.1\,10.0.0.2,21.0.0.1\,10.0.0.6,4789,33,,8
71,55.059999000,ICMP,21.0.0.1\,10.0.0.6,20.0.0.1\,10.0.0.2,4789,33,,0
72,56.044833000,ICMP,20.0.0.1\,10.0.0.2,21.0.0.1\,10.0.0.6,4789,33,,0
73,56.058357000,ICMP,21.0.0.1\,10.0.0.6,20.0.0.1\,10.0.0.2,4789,33,,0
74,56.177654000,ARP,20.0.0.1,239.1.1.6,4789,66,11.0.0.1,11.0.0.3,
75,56.183085000,ARP,20.0.0.2,20.0.0.1,4789,66,11.0.0.3,11.0.0.1,

```

Figure 36: IGMP, ARP and ICMP traffic encapsulated with VXLAN.

For traffic from b1 to b3, the outer IP header used $20.0.0.1 \rightarrow 20.0.0.2$, while the inner IP packet used $10.0.0.1 \rightarrow 10.0.0.3$. For traffic from b2 to b6, the outer path changed to $20.0.0.1 \rightarrow 21.0.0.1$, crossing R1. The VNI was 33 for **bluenet** and 66 for **rednet**.

During ARP resolution, the ARP Request was still a broadcast frame inside the encapsulated Ethernet payload, but the outer IP destination was the multicast group of the overlay: **239.1.1.3** for **bluenet** and **239.1.1.6** for **rednet**. The ARP Reply was then sent using unicast between the VTEPs, for example $20.0.0.2 \rightarrow 20.0.0.1$. When the overlay changes, the VNI and multicast group change, but the reply remains unicast between the VTEPs that learned each other through ARP. The ARP cache of the containers matched the MAC addresses observed in the corresponding ARP packets.

4.3.4 Comparison with AI explanation

The AI explanation correctly stated that VXLAN builds a Layer 2 overlay over a Layer 3 underlay and that the VNI separates overlay domains. The experiment added the operational detail required by the lab: with multicast VXLAN, the VTEPs depend on IGMP membership and multicast routing in the underlay. Without PIM dense mode on R1, traffic between **ub1/ub2** and **ub3** would not be forwarded.

4.4 Docker Swarm services

4.4.1 Swarm setup

The Docker Swarm experiment followed the service deployment exercise from the lab guide [3] and started from a clean configuration with the previous containers removed. The node `ub1` was used as the Swarm manager, while `ub2` and `ub3` were joined as workers. Two overlay networks were then created: `blue` and `red`.

Table 9: Docker Swarm roles used in the simulation.

Node	Address	Swarm role	Purpose
ub1	20.0.0.1/24	Manager	Service control and published port testing
ub2	20.0.0.2/24	Worker	Service task execution
ub3	21.0.0.1/24	Worker	Service task execution

```
# ub1
docker swarm init --advertise-addr 20.0.0.1
docker swarm join-token worker -q

# ub2 and ub3
docker swarm join --token <token> 20.0.0.1:2377

# ub1
docker network create --driver=overlay --attachable blue
docker network create --driver=overlay --attachable red
```

Listing 63: Swarm initialization and overlay network creation.

4.4.2 Service creation

The first service, `bluesvc`, was created in the `blue` overlay network with five replicas. The service published port 3000 externally and forwarded it to TCP port 80 inside the service containers.

```
docker service create --network blue --name bluesvc --replicas 5 \
--publish 3000:80 ruivaladas/ws-simple-2
```

Listing 64: Creation of the `bluesvc` Docker Swarm service.

Output - Docker Swarm: cria??o e distribui??o do bluesvc

```
> docker service rm bluesvc 2>/dev/null || true
> docker service create --network blue --name bluesvc --replicas 5 --publish 3000:80 ruivaladas/ws-simple-2
> docker service ps bluesvc --no-trunc
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS	
vuyr9h157yry	bluesvc	replicated	5/5	ruivaladas/ws-simple-2:latest	*:3000->80/tcp	

ID	NAME	IMAGE	ERROR	PORTS	NODE
DESIRED STATE	CURRENT STATE				
z48nwyhucejg2anaibt4s4i3	bluesvc.1	Running			ub3
Running	25 seconds ago				
qama36z9t3dvtqebnh0s3rsbn	bluesvc.2	Running			ub1
Running	27 seconds ago				
r7b4evc8gdk3legr0rnzmoiwb	bluesvc.3	Running			ub2
Running	25 seconds ago				
ugu9xmoe9k2dq5mb15x33ft3	bluesvc.4	Running			ub3
Running	25 seconds ago				
xeif1oc8rkig0bafpogol2w59	bluesvc.5	Running			ub2
Running	25 seconds ago				
ub3bluesvc.1	Running	26 seconds ago			
ub1 _ bluesvc.2	Running	27 seconds ago			
ub1 _ bluesvc.2	Running	27 seconds ago			
ub1 _ bluesvc.2	Running	27 seconds ago			
ub2 _ bluesvc.3	Running	25 seconds ago			
ub2 _ bluesvc.3	Running	25 seconds ago			
ub3bluesvc.4	Running	26 seconds ago			
ub2 _ bluesvc.5	Running	25 seconds ago			
ub2 _ bluesvc.5	Running	25 seconds ago			
You've hit 97b607f15055					
You've hit da4a7b3327cd					
You've hit 97b607f15055					

Figure 37: Creation of bluesvc, task distribution and initial HTTP responses.

The output in Figure 37 shows that the service was created and that Swarm distributed the service tasks across the available nodes. The HTTP response includes the container hostname, which made it possible to observe which replica answered each request.

4.4.3 Self-healing and worker recovery

To test self-healing, one service container running on a worker node was stopped manually. The manager marked the old task as failed and created a replacement task to maintain the desired number of replicas. When the stopped container was started manually, it was not reattached to the service as the valid task; Swarm kept the newly created task as part of the desired state.

The same behaviour was observed when service containers were removed on different nodes during the final cleanup. Swarm recreated tasks until the service converged back to the configured replica count.

The worker recovery test was performed by forcing ub3 to leave the Swarm and then joining it again with the worker token. After the rejoin, the manager showed a new entry for ub3; the old entry remained down until it was removed. After removing the old node entry and cleaning failed tasks, the service converged again with replicas distributed across ub1, ub2 and ub3.

```

===== manager after one task stopped =====
bash <<'EOF'

> docker service ps bluesvc --no-trunc

> docker service ls

> EOF

ID                                NAME                                IMAGE                                ...
z48nwyhucejg2anaibt4s4i3         bluesvc.1                          ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9df1ca85b2000695804f6a ...
qama36z9t3dvteqbnohs3rsbn        bluesvc.2                          ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9df1ca85b2000695804f6a ...
r7b4evc8gdk3legr0rnzmoiyb        bluesvc.3                          ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9df1ca85b2000695804f6a ...
ugu9xmoe9k2dq5mb15x33ft3         bluesvc.4                          ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9df1ca85b2000695804f6a ...
epas955a2cwcxllpgjmlumz5         bluesvc.5                          ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9df1ca85b2000695804f6a ...
xeif1oc8rkigobafpogo12w59        \_ bluesvc.5                      ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9df1ca85b2000695804f6a ...

ID                                NAME                                MODE                                REPLICAS                                IMAGE                                PORTS
vuyr9h157yry                     bluesvc                             replicated                         5/5                                     ruivaladas/ws-simple-2:latest      *:3000->80/tcp

root@ub1:/home/ubuntu# printf '\nHEND1779981184238:%s\n' $?

===== ub3 rejoin =====
bash <<'EOF'

> docker swarm join --token SWMTKN-1-10pgowp5545hmm2qf7ab84ezsvzu529o1m3q5ypajnxamu8bk-29418m879i1wasgpes731ouc2 20.0.0.1:2377

> docker info --format "node={{.Name}} swarm={{.Swarm.LocalNodeState}}"

> EOF

This node joined a swarm as a worker.

node=ub3 swarm=active

root@ub1:/home/ubuntu# printf '\nHEND1779981195812:%s\n'
===== manager after ub3 rejoin =====
bash <<'EOF'

> docker node ls

> docker service ps bluesvc --no-trunc

> EOF

ID                                HOSTNAME                                STATUS                                AVAILABILITY                                MANAGER STATUS                                ENGINE VERSION
lvj2370r2ir9h9h2v35azsm2q *      ub1                                    Ready                                Active                                Leader                                         20.10.12
jgg3ov8nhfft076ekq28tvoem        ub2                                    Ready                                Active                                                                        20.10.12
cik665fzowwanszs2cfmj7rcj        ub3                                    Ready                                Active                                                                        20.10.12
seud2bfgz8shl2pplji7tkn47        ub3                                    Down                                Active                                                                        20.10.12

ID                                NAME                                IMAGE                                ...
vt62sco8d7htvpzwbgn2dmos         bluesvc.1                          ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9df1ca85b2000695804f6a ...
z48nwyhucejg2anaibt4s4i3         \_ bluesvc.1                      ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9df1ca85b2000695804f6a ...
qama36z9t3dvteqbnohs3rsbn        bluesvc.2                          ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9df1ca85b2000695804f6a ...
r7b4evc8gdk3legr0rnzmoiyb        bluesvc.3                          ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9df1ca85b2000695804f6a ...
ytdylv3bcs4317d5s3cuc25jt        bluesvc.4                          ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9df1ca85b2000695804f6a ...
ugu9xmoe9k2dq5mb15x33ft3         \_ bluesvc.4                      ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9df1ca85b2000695804f6a ...
epas955a2cwcxllpgjmlumz5         bluesvc.5                          ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9df1ca85b2000695804f6a ...
xeif1oc8rkigobafpogo12w59        \_ bluesvc.5                      ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9df1ca85b2000695804f6a ...

root@ub1:/home/ubuntu# printf '\nHEND1779981205022:%s\n' $?

HEND1779981205022:0

root@ub1:/home/ub

```

Figure 38: Self-healing behaviour after stopping a task and worker recovery after rejoining the Swarm.

After node recovery, task redistribution can be forced with commands such as `docker node update --availability drain/active <node>` or `docker service update --force bluesvc`. This recreates tasks and allows the scheduler to place them again across the available nodes.

4.4.4 Load balancing

The published service was reachable through any Swarm node: `20.0.0.1:3000`, `20.0.0.2:3000` and `21.0.0.1:3000`. This shows that the published port is handled by the Docker Swarm routing mesh, not only by the manager node.

Output - Load balancing, VNIs e interfaces

```
> echo '--- requests via 20.0.0.1:3000 ---'
> echo '--- requests via 20.0.0.2:3000 ---'
> echo '--- requests via 21.0.0.1:3000 ---'
> docker network inspect ingress blue --format '{{.Name}} {{index .Options "com.docker.network.driver.overlay.vxlanid_list"}} {{range .IPAM.Config}}{{.Subnet}} {{end}}'
> for c in $(docker ps --filter name=bluesvc -q); do echo container=$c name=$(docker inspect -f '{{.Name}}' $c); pid=$(docker inspect -f '{{.State.Pid}}' $c); nsenter -t $pid -n ip -br addr; done
--- requests via 20.0.0.1:3000 ---
You've hit 7d26a39fa30d
You've hit f780fe93eff5
You've hit bcf3179d7df8
You've hit da4a7b3327cd
You've hit c7a6dffbcd70
You've hit 7d26a39fa30d
You've hit f780fe93eff5
You've hit bcf3179d7df8
You've hit da4a7b3327cd
You've hit c7a6dffbcd70
--- requests via 20.0.0.2:3000 ---
You've hit c7a6dffbcd70
You've hit 7d26a39fa30d
You've hit f780fe93eff5
You've hit bcf3179d7df8
You've hit da4a7b3327cd
--- requests via 21.0.0.1:3000 ---
You've hit 7d26a39fa30d
You've hit f780fe93eff5
You've hit bcf3179d7df8
You've hit da4a7b3327cd
You've hit c7a6dffbcd70
ingress 4096 10.0.0.0/24
blue 4097 10.0.1.0/24
red 4098 10.0.2.0/24
container=bcf3179d7df8 name=/bluesvc.5.epas955a2cudcx1lpgjnlumz5
```

Figure 39: Load balancing tests, overlay VNIs and local container interfaces.

A Python client was then executed on `ub1` and sent 12 HTTP requests to `20.0.0.1:3000`. The responses were cyclic and balanced: each of the three final replicas answered four times.

```
==== Python client bluesvc 12 requests ====
01 You've hit d8543f833c3c
02 You've hit e254d47ef241
03 You've hit 7c2817949998
04 You've hit d8543f833c3c
05 You've hit e254d47ef241
06 You've hit 7c2817949998
07 You've hit d8543f833c3c
08 You've hit e254d47ef241
09 You've hit 7c2817949998
10 You've hit d8543f833c3c
11 You've hit e254d47ef241
12 You've hit 7c2817949998
summary
4 You've hit 7c2817949998
4 You've hit d8543f833c3c
4 You've hit e254d47ef241
```

Figure 40: Python client output used to infer the Swarm service load-balancing behaviour.

The observed behaviour is compatible with a round-robin distribution across healthy service tasks.

4.4.5 Scale down and container interfaces

The service was scaled down to three replicas:

```
docker service scale bluesvc=3
```

Listing 65: Command used to scale bluesvc down to three replicas.

After cleanup of failed tasks produced during the self-healing tests, the service converged to 3/3 replicas.

Output - Scale down final: 3/3 replicas						
ID	NAME	MODE	REPLICAS	IMAGE	PORTS	
vuyr9hi157yry	bluesvc	replicated	3/3	ruivaladas/ws-simple-2:latest	*:3000->80/tcp	
twvx1m4uo165nw1i5ua3tub1c	bluesvc.1	Running		ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9fdf1ca85b2000695804f6a		
ub3		Running		Running 49 seconds ago		
vt62sco8d7htvpzwbgn2dmos	\ bluesvc.1	Running		ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9fdf1ca85b2000695804f6a		
ub3		Shutdown		Failed 55 seconds ago "task: non-zero exit (137)"		
z48nwxyhucejg2anaibt4s4i3	\ bluesvc.1	Running		ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9fdf1ca85b2000695804f6a		
seud2bfgz8sh12pplji7tkn47	Shutdown			Orphaned 5 seconds ago		
m7ekaanjvep7u5q4se8kt9zbi	bluesvc.2	Running		ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9fdf1ca85b2000695804f6a		
ub1		Running		Running 53 seconds ago		
qama36z9t3dvtqbnh0s3rsbn	\ bluesvc.2	Running		ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9fdf1ca85b2000695804f6a		
ub1		Shutdown		Failed 59 seconds ago "task: non-zero exit (137)"		
qvfvqqvqpglijltetd9olt9j7	bluesvc.3	Running		ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9fdf1ca85b2000695804f6a		
ub2		Running		Running 51 seconds ago		
r7b4evc8gdk3legr0rnzmoiwb	\ bluesvc.3	Running		ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9fdf1ca85b2000695804f6a		
ub2		Shutdown		Failed 57 seconds ago "task: non-zero exit (137)"		
ugufxmooe9k2dq5mb15x33ft3	bluesvc.4	Running		ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9fdf1ca85b2000695804f6a		
seud2bfgz8sh12pplji7tkn47	Shutdown			Orphaned 5 seconds ago		

Figure 41: Final state of bluesvc after scaling down to three replicas.

Docker Swarm does not provide native metric-based autoscaling equivalent to the Kubernetes Horizontal Pod Autoscaler. In this experiment, scaling was manual; automatic scaling would require external logic or additional tooling.

Each service container had three relevant interfaces besides the loopback interface: one in the **ingress** network, one in **docker_gwbridge**, and one in the application overlay network **blue**.

===== bluesvc container interfaces after scale to 3 replicas =====	
[ub1]	container=7c2817949998 name=/bluesvc.2.m7ekaanjvep7u5q4se8kt9zbi
eth1@if50	UP 10.0.0.15/24
eth2@if52	UP 172.18.0.3/16
eth0@if54	UP 10.0.1.15/24
eth1@if50:	MAC 02:42:0a:00:00:0f
eth2@if52:	MAC 02:42:ac:12:00:03
eth0@if54:	MAC 02:42:0a:00:01:0f
[ub2]	container=e254d47ef241 name=/bluesvc.3.qvfvqqvqpglijltetd9olt9j7
eth1@if57	UP 10.0.0.16/24
eth2@if59	UP 172.18.0.3/16
eth0@if61	UP 10.0.1.17/24
eth1@if57:	MAC 02:42:0a:00:00:10
eth2@if59:	MAC 02:42:ac:12:00:03
eth0@if61:	MAC 02:42:0a:00:01:11
[ub3]	container=d8543f833c3c name=/bluesvc.1.twvx1m4uo165nw1i5ua3tub1c
eth1@if57	UP 10.0.0.17/24
eth2@if59	UP 172.18.0.3/16
eth0@if61	UP 10.0.1.19/24
eth1@if57:	MAC 02:42:0a:00:00:11
eth2@if59:	MAC 02:42:ac:12:00:03
eth0@if61:	MAC 02:42:0a:00:01:13

Figure 42: IP and MAC addresses of bluesvc containers on the three Swarm nodes.

The overlay VNIs observed in the simulation were:

Table 10: VXLAN VNIs used by Docker Swarm overlay networks.

Network	VNI	Purpose
ingress	4096	Routing mesh for published ports
blue	4097	Application overlay for bluesvc
red	4098	Application overlay for redsvc

4.4.6 Routing mesh packet path

A Wireshark capture was performed on the **ub1** physical interface while the Python client sent HTTP requests to **20.0.0.1:3000**. This corresponds to the path-tracing test requested in the lab: the client contacts the published port of the node **ub1**, while Swarm selects one of the three service replicas. The capture shows that the routing mesh uses the **ingress** overlay network over VXLAN.

Wireshark - Docker Swarm routing mesh sobre VXLAN ingress

```
frame.number,frame.time_relative,ws.col.protocol,ip.src,ip.dst,tcp.srcport,tcp.dstport,udp.dstport,vxlan.vni
255,28.892843000,TCP,20.0.0.1\,10.0.0.2,21.0.0.1\,10.0.0.14,42006,3000,4789,4096
256,28.913435000,TCP,21.0.0.1\,10.0.0.14,20.0.0.1\,10.0.0.2,3000,42006,4789,4096
257,28.914429000,TCP,20.0.0.1\,10.0.0.2,21.0.0.1\,10.0.0.14,42006,3000,4789,4096
258,28.915494000,HTTP,20.0.0.1\,10.0.0.2,21.0.0.1\,10.0.0.14,42006,3000,4789,4096
259,28.944420000,TCP,21.0.0.1\,10.0.0.14,20.0.0.1\,10.0.0.2,3000,42006,4789,4096
262,29.048702000,TCP,21.0.0.1\,10.0.0.14,20.0.0.1\,10.0.0.2,3000,42006,4789,4096
263,29.049202000,TCP,20.0.0.1\,10.0.0.2,21.0.0.1\,10.0.0.14,42006,3000,4789,4096
264,29.059313000,HTTP,21.0.0.1\,10.0.0.14,20.0.0.1\,10.0.0.2,3000,42006,4789,4096
265,29.060319000,TCP,20.0.0.1\,10.0.0.2,21.0.0.1\,10.0.0.14,42006,3000,4789,4096
266,29.066828000,TCP,20.0.0.1\,10.0.0.2,21.0.0.1\,10.0.0.13,42010,3000,4789,4096
267,29.079292000,TCP,21.0.0.1\,10.0.0.14,20.0.0.1\,10.0.0.2,3000,42006,4789,4096
268,29.089534000,TCP,21.0.0.1\,10.0.0.13,20.0.0.1\,10.0.0.2,3000,42010,4789,4096
269,29.090298000,TCP,20.0.0.1\,10.0.0.2,21.0.0.1\,10.0.0.13,42010,3000,4789,4096
270,29.090298000,HTTP,20.0.0.1\,10.0.0.2,21.0.0.1\,10.0.0.13,42010,3000,4789,4096
271,29.123262000,TCP,21.0.0.1\,10.0.0.13,20.0.0.1\,10.0.0.2,3000,42010,4789,4096
272,29.217621000,TCP,21.0.0.1\,10.0.0.13,20.0.0.1\,10.0.0.2,3000,42010,4789,4096
273,29.218364000,TCP,20.0.0.1\,10.0.0.2,21.0.0.1\,10.0.0.13,42010,3000,4789,4096
274,29.226024000,HTTP,21.0.0.1\,10.0.0.13,20.0.0.1\,10.0.0.2,3000,42010,4789,4096
275,29.227024000,TCP,20.0.0.1\,10.0.0.2,21.0.0.1\,10.0.0.13,42010,3000,4789,4096
276,29.248316000,TCP,21.0.0.1\,10.0.0.13,20.0.0.1\,10.0.0.2,3000,42010,4789,4096
277,29.348286000,TCP,20.0.0.1\,10.0.0.2,20.0.0.2\,10.0.0.7,42034,3000,4789,4096
278,29.350408000,TCP,20.0.0.2\,10.0.0.7,20.0.0.1\,10.0.0.2,3000,42034,4789,4096
279,29.350911000,TCP,20.0.0.1\,10.0.0.2,20.0.0.2\,10.0.0.7,42034,3000,4789,4096
280,29.351433000,HTTP,20.0.0.1\,10.0.0.2,20.0.0.2\,10.0.0.7,42034,3000,4789,4096
281,29.354780000,TCP,20.0.0.2\,10.0.0.7,20.0.0.1\,10.0.0.2,3000,42034,4789,4096
282,29.456643000,TCP,20.0.0.2\,10.0.0.7,20.0.0.1\,10.0.0.2,3000,42034,4789,4096
283,29.457143000,HTTP,20.0.0.2\,10.0.0.7,20.0.0.1\,10.0.0.2,3000,42034,4789,4096
```

Figure 43: Routing mesh traffic transported over the ingress VXLAN network.

When the selected replica was located on `ub1`, the request entered the routing mesh and was delivered locally. The HTTP response returned directly through the same node. When the selected replica was located on `ub2` or `ub3`, the request was encapsulated in UDP/4789 with VNI 4096 and sent through the physical underlay to the remote node. The HTTP response then followed the reverse path through the same `ingress` overlay.

For tunneled packets, the outer IP and MAC addresses belonged to the physical host interfaces, for example `20.0.0.1 → 20.0.0.2` or `20.0.0.1 → 21.0.0.1`. The inner IP and MAC addresses belonged to the `ingress` interfaces of the service containers.

4.4.7 redsvc service

A second service was created in the `red` overlay network. It used three replicas and published port 3001.

```
docker service create --network red --name redsvc --replicas 3 \
--publish 3001:80 ruivaladas/ws-simple-2
```

Listing 66: Creation of the redsvc Docker Swarm service.

Output - redsvc na rede red

```
> docker service rm redsvc 2>/dev/null || true
> docker service create --network red --name redsvc --replicas 3 --publish 3001:80 ruivaladas/ws-simple-2
> docker service ps redsvc --no-trunc
> echo '--- redsvc requests via published port ---'
> docker network inspect ingress blue --format '{{.Name}}' '{{index .Options "com.docker.network.driver.overlay.vxlanid_list"}}' '{{range .IPAM.Config}}{{.Subnet}}{{end}}'
tvtj8bwkgh74 redsvc replicated 3/3 ruivaladas/ws-simple-2:latest *:3001->80/tcp
2ilr4o72e89vj7f4k0qnssbm3 redsvc.1 ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9fd1ca85b2000695804f6a ub2
Running 25 seconds ago
q36trje3b53jzq8lauqm0l0r redsvc.2 ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9fd1ca85b2000695804f6a ub3
Running 26 seconds ago
1jj584wjlveykbt44d8fy317 redsvc.3 ruivaladas/ws-simple-2:latest@sha256:2b95be57f92735a8559ff3f66e075c9532af4d04a9fd1ca85b2000695804f6a ub1
Running 25 seconds ago
--- redsvc requests via published port ---
You've hit 9121a53c69c3
You've hit e8ba2a1d913f
You've hit c6b54d6ba1b6
You've hit 9121a53c69c3
You've hit e8ba2a1d913f
You've hit c6b54d6ba1b6
ingress 4096 10.0.0.0/24
blue 4097 10.0.1.0/24
red 4098 10.0.2.0/24
```

Figure 44: Creation and validation of redsvc in the red overlay network.

The service was reachable through 20.0.0.1:3001, and the responses alternated between the three replicas. The main difference compared with `bluesvc` was the application overlay: `redsvc` used the `red` overlay with VNI 4098, while `bluesvc` used the `blue` overlay with VNI 4097. Published ports continued to use the `ingress` overlay with VNI 4096.

4.4.8 Comparison with AI explanation

The AI explanation of Docker Swarm services was correct in stating that the manager maintains the desired state, redistributes tasks and exposes services through the routing mesh. The experimental results added three practical observations. First, stopping a service container does not change the desired state; it causes Swarm to create a replacement task. Second, a worker that leaves and rejoins may appear as a new node, so the old down entry must be removed to clean the Swarm state. Third, the application overlay network, such as `blue` or `red`, is separate from the `ingress` overlay used for published ports.

4.4.9 Conclusion

The Docker Swarm experiment confirmed that a Swarm service is a replicated application object managed by the Swarm control plane. The manager keeps the desired number of replicas, replaces failed tasks, distributes requests through the routing mesh, and isolates services with overlay networks. The `blue` and `red` overlays isolated application traffic, while the `ingress` overlay carried traffic for published ports.

References

- [1] Valadas, R. (2026). *Lab guide no. 2.1: IPSec and DMVPN* [Laboratory guide]. Instituto Superior Tecnico.
- [2] Valadas, R. (2026). *Lab guide no. 2.2: VRFs and BGP-MPLS L3VPNs* [Laboratory guide]. Instituto Superior Tecnico.
- [3] Valadas, R. (2026). *Lab guide no. 2.3: Network Containers* [Laboratory guide]. Instituto Superior Tecnico.
- [4] Valadas, R. (2026). *VRFs and BGP-MPLS L3VPNs* [SAAR course slides]. Instituto Superior Tecnico.